

ADAS 前视视频车道线与车辆检测跟踪技术报告

日期：2026-05-25
姓名：师文翔
项目：前置车载摄像头道路特征检测
输入视频：project_video.mp4
最终输出视频：outputs/final_yolop_lane_yolo_bytetrack_p63.mp4

摘要

本项目面向前视车载摄像头视频，实现车道边界检测、周围车辆检测与车辆跟踪。系统最终采用 YOLOP 神经网络进行车道线分割，采用 YOLO11 与 ByteTrack 进行车辆检测和跟踪，并通过中近距离裁剪策略抑制远端车道线分割的离散误检。完整视频共 1260 帧，输出视频保持 1280x720 分辨率和 25 FPS，车道线检测覆盖全部帧，车辆检测覆盖 1163 帧，共检测车辆 1935 次，并产生 26 个唯一跟踪 ID。

1. 任务要求

作业要求从前置车载摄像头视频中检测道路特征，包括：

- 1. 识别视频中的车道边界。
- 2. 用目标框标出道路中出现的车辆。
- 3. 对周围车辆进行跟踪。
- 4. 附加估计车辆在车道内的位置，给出车辆中心距左右车道边界的距离。

本项目在 RTX 4090 本地环境上完成模型部署、视频处理、关键帧检查和结果验证。

2. 方法调研与方案比较

项目调研和尝试了传统视觉方法、神经网络车道线方法、YOLO 目标检测跟踪方法以及 SAM/SAM2 分割方法。

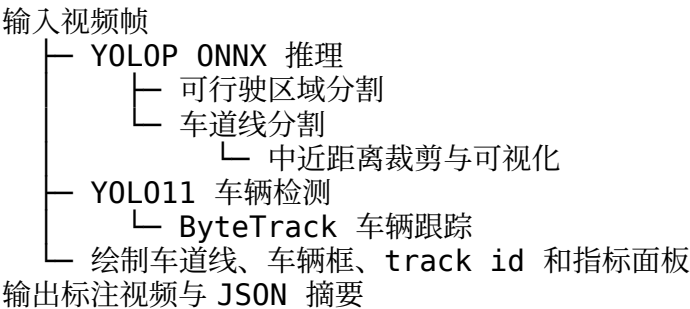
方法	适用模块	优点	实验观察	结论
Hough / 颜色边缘检测	车道线	实现简单，速度快	对光照、阴影和弯道较敏感，远近场稳定性不足	作为调试基线，不作为最终方案
透视变换 + 多项式拟合	车道线	可输出连续曲线和曲率	对当前视频部分帧可用，但依赖强几何先验，容易出现不自然拟合	效果不如神经网络分割稳定，未采用
YOLOP	车道线	可直接输出车道线和可行驶区域 mask	车道线整体识别精确，局部远端存在少量	采用为最终车道线方案

YOLO11 + ByteTrack	车辆检测与跟踪	检测和跟踪流程成熟，速度较好	离散 outlier 车辆框和 ID 稳定，满足作业需求	采用为最终车辆方案
SAM/SAM2	车辆 mask refinement	车辆轮廓分割更细	需要检测框或点作为 prompt，不适合作为车辆类别检测主入口	作为对比方案，不作为最终主链路

综合比较后，最终系统使用 YOLOP 负责车道线感知，使用 YOLO11 + ByteTrack 负责车辆检测与跟踪。

3. 系统总体结构

最终系统按帧处理输入视频，流程如下：



系统实现为本地 Python 项目，主要依赖如下：

- OpenCV：视频读写、图像后处理与可视化。
- ONNXRuntime GPU：运行 YOLOP ONNX 模型。
- Ultralytics：运行 YOLO11 检测和 ByteTrack 跟踪。
- PyTorch CUDA 12.8：支撑 Ultralytics 模型推理。

4. 车道线检测实现

4.1 YOLOP 神经网络分割

车道线检测采用 YOLOP 的 yolop-640-640.onnx 模型。模型输出包括：

- drive_area_seg：可行驶区域。
- lane_line_seg：车道线区域。

输入预处理步骤为：

1. 将原始 1280x720 帧按比例缩放并 letterbox 到 640x640。
2. OpenCV 读取的 BGR 图像转换为 RGB。
3. 像素归一化到 [0, 1]。
4. 使用 ImageNet 均值和方差标准化。

5. 转换为 NCHW 格式送入 ONNXRuntime。

实际实验中，RGB 输入比直接使用 BGR 输入得到的车道线 mask 更完整。

4.2 车道线后处理

YOLOP 的车道线输出已经较为准确，但远端区域偶尔会出现离散 outlier。考虑到驾驶辅助系统更关注车辆前方中近距离区域，本项目在可视化阶段加入显示裁剪窗口。

最终采用的裁剪版本为 p63：

- 仅显示图像高度 63% 以下的车道线 overlay。
- 梯形窗口顶部范围为图像宽度 40% 到 60%。
- 梯形窗口底部范围为图像宽度 16% 到 90%。

该策略保留车辆前方可操作范围内的车道边界，同时隐藏远端不稳定分割点。模型仍对整帧进行推理，裁剪只影响最终显示效果。

4.3 车辆在车道内的位置估计

为了估计车辆中心到左右车道边界的距离，系统在图像中下部稳定区域从画面中心向左右寻找最近的 lane cluster，将其视为本车道左右边界。随后假设标准车道宽度为 3.7 米，计算像素到米的横向比例：

$xm_per_pix = 3.7 / lane_width_px$

其中 lane_width_px 为左右车道边界的像素间距。图像中心点近似为车辆中心点，因此：

```
left_distance_m = (center_x - left_lane_x) * xm_per_pix
right_distance_m = (right_lane_x - center_x) * xm_per_pix
offset_m = (center_x - lane_center_x) * xm_per_pix
```

由于未提供相机标定参数，该距离为单目近似估计，适合作业展示和相对位置分析。

5. 车辆检测与跟踪实现

车辆检测采用 Ultralytics YOLO11，检测类别限定为 COCO 数据集中的车辆相关类别：

- car
- motorcycle
- bus
- truck

跟踪部分采用 ByteTrack。ByteTrack 基于连续帧检测框进行数据关联，能够为同一车辆分配稳定的 track id。

最终视频中车辆标注包含：

- 车辆类别。

- track id。
- 检测置信度。
- 车辆目标框。

实验表明，YOLO11 + ByteTrack 对本视频中的车辆检测和跟踪已经足够稳定，因此最终主链路未加入 SAM/SAM2，以保持系统简单、清晰和可复现。

6. SAM/SAM2 对比

项目实现了 SAM/SAM2 对比工具，用 YOLO 检测框作为 prompt 生成车辆 mask，并额外验证了 SAM2 video propagation。对比结论如下：

- SAM/SAM2 能获得更细的车辆轮廓 mask。
- SAM/SAM2 本身不是车辆类别检测器，仍需要 YOLO 框作为 prompt。
- 对本作业要求的“车辆框检测 + 跟踪”而言，YOLO11 + ByteTrack 更直接。
- SAM2 适合作为后续实例分割增强模块，而不是最终主检测链路。

因此最终提交视频采用 YOLO11 + ByteTrack 进行车辆检测与跟踪。

7. 实验结果

最终输出文件：

- 标注视频：outputs/final_yolop_lane_yolo_bytetrack_p63.mp4
- 指标摘要：outputs/final_yolop_lane_yolo_bytetrack_p63.json
- 关键帧样张：outputs/final_yolop_lane_yolo_samples_p63/

完整视频处理结果如下：

指标

输入帧数
输出帧数
视频分辨率
输入帧率
视频时长
车道线检测帧数
车辆检测帧数
车辆检测总数
唯一 track id 数
处理吞吐

最后一帧的车道位置估计如下：

字段

left_detected

right_detected
offset_m
left_distance_m
right_distance_m
curvature_m

本方案没有输出曲率值，原因是最终路线没有采用多项式拟合，而是以神经网络 segmentation 为主。为了避免引入不稳定的曲线拟合，报告保留车辆相对车道位置和左右边界距离作为主要车道内位置指标。

8. 验证

本地单元测试和流水线测试命令：

```
uv run pytest -q
```

结果：

4 passed

视频属性检查命令：

```
ffprobe -v error -select_streams v:0 \
  -show_entries stream=width,height,nb_frames,duration,r_frame_rate \
  -of json outputs/final_yolop_lane_yolo_bytetrack_p63.mp4
```

结果表明最终视频为：

- 1280x720。
- 1260 帧。
- 25 FPS。
- 50.4 秒。

同时检查了首帧、中间帧和尾帧样张：

- outputs/final_yolop_lane_yolo_samples_p63/frame_0000.jpg
- outputs/final_yolop_lane_yolo_samples_p63/frame_0630.jpg
- outputs/final_yolop_lane_yolo_samples_p63/frame_1259.jpg

9. 总结

本项目完成了前视道路视频中的车道线检测、车辆检测与车辆跟踪。最终方案使用 YOLOP 进行车道线神经网络分割，并通过 p63 中近距离裁剪减少远端 outlier 对视觉效果的影响；车辆模块采用 YOLO11 + ByteTrack，能够稳定输出车辆框和跟踪 ID。

整体系统满足作业要求，并在检测质量、视觉稳定性、运行速度和实现复杂度之间取得了较好的平衡。